利用 dot 与 fiber 表达 Noninterference 的类型

类型定义、Denotational 语义与 Hypra 例子

2026 年 10 月 15 日

摘要

本文利用显式观察支持与 fiber 语义，在 Context Types 中描述 noninterference。在 refinement、函数箭头与 persistence 类型的基础上，引入整个类型的观察构造 $\mathsf{Observe}(X, \tau)$，并分别给出类型语法、良构条件和 $[\![\tau]\!]_\Sigma : \mathrm{Term} \to \mathrm{Formula}$ 的递归解释。派生结果类型 $\mathsf{NI}_b(l)$ 要求固定公开输入后，全部返回结果具有一致取值。整数程序 `secure` 与其泄漏变体 `first_if` 展示了这一性质与跨分支路径 VC 的联系。结果 capability 上的性质、辅助契约的准确性和路径 VC 的正确性已在 Rocq 中证明，完整程序项类型解释的桥接条件与实现范围在文中单独说明。

1 目标、范围与符号

本文只观察全定义、确定性整数程序的最终返回值。秘密输入为 h，公开输入为 l。程序中的输出或中间变量记为 o；类型解释中的新鲜结果变量记为 ν。目标为

$$\forall h_1, h_2, l \in \mathbb{Z}. \quad f(h_1, l) = f(h_2, l). \tag{1}$$

允许输入 capability 存在相关性，不预设 high/low 为笛卡尔积。该性质允许公开输入不同的执行返回不同结果。

类型记号沿用论文的 $\{\nu : b \mid \phi\}$、$[\nu : b \mid \phi]$、$\to$、$-\!\!*$、$\sqcap$、$\oplus$ 与 $\square$；逻辑记号沿用 $\boxed{\mathrm{o}}P$、$\boxed{\mathrm{u}}P$、$X \triangleright P$ 与支持域扩充文档中的 $P \blacktriangleright Q$、$X.\{\phi\}$。$\mathsf{Observe}$ 是 Context Types 的语法扩展；Low_b、$\mathsf{NI}_b(l)$ 均为派生类型缩写。$\mathsf{Low}(\nu)$ 则是已有逻辑层的取值一致性公式，两者不属于同一语法类别。

2 Type Syntax

2.1 基础类型、qualifier 与 context type

基础类型与基本函数类型采用 Context Types 的分类 [1]：

$$b ::= \mathsf{unit} \mid \mathsf{bool} \mid \mathsf{nat} \mid \mathsf{int} \mid \cdots, \qquad s ::= b \mid s \to s.$$

int 用于整数程序实例。Rocq CoreLang 的实现范围见第 8 节。

qualifier 的基础语法沿用 Context Types Figure 2，另加入支持域扩充文档的 dot[2]：

$$\phi ::= v \mid x \mid \mathsf{op}\ \overline{v} \mid \bot \mid \top \mid \neg\phi \mid \phi \wedge \phi \mid \phi \vee \phi \mid \phi \Rightarrow \phi$$
$$\mid \forall x : b.\phi \mid \exists x : b.\phi \mid X.\{\phi\}.$$

X 是有限变量集。artifact 实际用带显式支持域的语义谓词表示 qualifier，而非上述一阶语法树。

定义 2.1 (Context type syntax)**.** 采用 *Context Types Figure 2* 的类型构造及其 *Section 5.1* 的 *persistence* 类型：

$$\begin{aligned}\tau ::= {} & \{\nu : b \mid \phi\} \mid [\nu : b \mid \phi] \mid \tau \sqcap \tau \mid \tau \oplus \tau \\ & \mid x : \tau \to \tau \mid x : \tau \mathrel{-\!\!*} \tau \mid \Box\tau \mid \mathsf{Observe}(X, \tau).\end{aligned} \tag{2}$$

$\mathsf{Observe}(X, \tau)$ 扩展了原 *context-type* 语法，其余构造沿用 *Context Types*。[1] ν 在基础 *refinement* 内绑定；x 在两个函数类型的返回类型内绑定。X 是被观察的自由变量集合，不绑定新变量。

定义 2.2 (Type context syntax)**.** 上下文语法保持论文原样：

$$\Gamma ::= \varnothing \mid x : \tau \mid \Gamma, \Gamma \mid \Gamma * \Gamma \mid \Gamma \oplus \Gamma.$$

Γ 表示类型上下文；其 *denotational* 语义在第 *4* 节定义。

2.2 Dot 的支持域与新增构造的良构条件

qualifier 的声明支持是 $\mathrm{FV}(\phi)$。按支持域扩充定义，

$$\mathrm{FV}(X.\{\phi\}) = X \cup \mathrm{FV}(\phi).$$

在恰好支持域的环境约定下，σ 的域预先限定为 $X \cup \mathrm{FV}(\phi)$，于是

$$\sigma \models X.\{\phi\} \quad \Longleftrightarrow \quad (\sigma \restriction_{\mathrm{FV}(\phi)}) \models \phi. \tag{3}$$

若允许任意较大的环境，完整条件改写为

$$\sigma \models X.\{\phi\} \Longleftrightarrow (X \cup \mathrm{FV}(\phi)) \subseteq \mathrm{dom}(\sigma) \wedge (\sigma \restriction_{\mathrm{FV}(\phi)}) \models \phi.$$

因此不能从右侧投影满足原谓词，自动推断新增观察变量已绑定。`qual_pad` 实现的是前一种带环境类型约束的版本。[2]

对新增的整个类型构造，规定

$$\begin{aligned}\lfloor \mathsf{Observe}(X, \tau) \rfloor &= \lfloor \tau \rfloor, \\ \mathrm{FV}(\mathsf{Observe}(X, \tau)) &= X \cup \mathrm{FV}(\tau).\end{aligned}$$

保留论文的 $\Sigma \vdash_{\mathrm{WF}} \tau$ 良构判断，并提出新规则

$$\frac{\Sigma \vdash_{\mathrm{WF}} \tau \qquad X \subseteq \mathrm{dom}(\Sigma)}{\Sigma \vdash_{\mathrm{WF}} \mathsf{Observe}(X, \tau)}.$$

该规则规定观察变量必须在基本类型上下文中已绑定。其 Rocq 集成状态见第 8 节。[3] 原函数类型的作用域照常处理：第二个参数和返回类型可以引用先前绑定的参数。

[1] Rocq 对应定义：`/home/sdywolf/Project/UnderLogicAndType/ContextTyping/theories/FiberResult.v`；第 10 行起；`RTObserve` 是结果类型原型；原生 `context_ty` 尚未加入该构造。

[2] Rocq 对应定义：`/home/sdywolf/Project/UnderLogicAndType/ContextQualifier/theories/SupportPadding.v`；第 11 行起；`qual_pad` 定义 qualifier 支持域扩充。

[3] Rocq 对应定义：`/home/sdywolf/Project/UnderLogicAndType/ContextTyping/theories/FiberResult.v`；第 75 行起；`fiber_result_denote` 对 `RTObserve` 给出结果层 fiber 解释；本文完整类型的 erasure 与良构规则尚未集成到原生类型定义。

2.3 派生类型

定义 2.3 (Low 与 NI 结果类型的缩写)**.**

$$\mathsf{Low}_b := \square\{\nu : b \mid \top\}, \tag{4}$$

$$\mathsf{NI}_b(l) := \mathsf{Observe}(\{l\}, \mathsf{Low}_b). \tag{5}$$

Low_b 使用 *persistence* 类型；$\mathsf{NI}_b(l)$ 使用整个类型的观察包装。两者均为式 (2) 中类型表达式的派生缩写。[4]

定义 2.4 (Noninterference 函数类型)**.** 记 $\tau_{\top,b} := \{\nu : b \mid \top\}$。为保持原程序参数顺序 h, l，定义 [5]

$$T_{\mathsf{NI}} := h : \tau_{\top,\mathsf{int}} \to l : \tau_{\top,\mathsf{int}} \to \mathsf{NI}_{\mathsf{int}}(l), \tag{6}$$

$$T_{\mathrm{pres}} := h : \tau_{\top,\mathsf{int}} \to l : \mathsf{Low}_{\mathsf{int}} \to \mathsf{Low}_{\mathsf{int}}. \tag{7}$$

[6] 两种函数类型均使用普通函数箭头，参数与返回类型按 *context-type* 语法组合。T_{NI} 允许不同低输入的结果不同；T_{pres} 检查低输入能力固定时是否保持结果固定。

T_{NI}：在每个公开输入 fiber 内约束输出。 两个参数的类型 $\tau_{\top,\mathsf{int}} = \{\nu : \mathsf{int} \mid \top\}$ 都允许任意整数取值。在一个输入 capability 中，h 和 l 均可变化；其 Over 解释不要求该集合覆盖所有整数。函数类型要求对所有满足参数类型的 capability 成立，因此其保证不限于某一个预先选择的输入集合。返回类型 $\mathsf{NI}_{\mathsf{int}}(l)$ 先固定公开输入 l，再在对应 fiber 中检查 $\mathsf{Low}_{\mathsf{int}}$：该 fiber 内的所有返回结果必须具有同一个整数值。秘密输入 h 在 fiber 内仍可变化，故这些变化不能引起输出变化。不同 l 的 fiber 可以有不同的返回值；输出依赖公开输入是允许的。

T_{pres}：保持公开输入能力的取值一致性。 第一个参数 h 的 Over 类型仍允许其变化。第二个参数具有 $\mathsf{Low}_{\mathsf{int}} = \square\{\nu : \mathsf{int} \mid \top\}$，要求参与这次类型检查的输入 capability 中，所有环境给 l 相同的值。这个值不需要在类型中预先指定：对于每个满足条件的 capability，它可以是任意整数 a。返回值也具有 $\mathsf{Low}_{\mathsf{int}}$，因此所有这些环境的返回结果必须一致，即使它们的秘密输入不同。另一个公开输入值 a' 对应的输入 capability 可以得到另一结果。这里的 persistence 表达一次 capability 内的取值一致性，不要求函数在所有公开输入上返回同一个常量。

两种表达的区别与联系。 T_{NI} 在同一个输入 capability 中按 l 分组；T_{pres} 则对公开输入已经一致的 capability 提出输出一致性要求。例如，对于返回 $2l$ 的函数，输入 $(h, l) = (1, 3), (-1, 3), (1, 4)$ 分别得到 $6, 6, 8$。T_{NI} 允许这个整体输入集合，因为 $l = 3$ 的 fiber 中输出均为 6，$l = 4$ 的 fiber 中输出为 8。对 T_{pres}，分别取公开输入为 3 和 4 的两个 capability 检查，结果各自一致；混合

[4] Rocq 对应定义：`/home/sdywolf/Project/UnderLogicAndType/ContextTyping/theories/LowContextType.v` 第 6 行的 `low_context_ty` 给出原生 persistence 类型；`/home/sdywolf/Project/UnderLogicAndType/ContextTyping/theories/FiberResult.v` 第 82 行的 `ni_result_ty` 给出 NI 结果类型原型。

[5] Rocq 对应定义：`/home/sdywolf/Project/UnderLogicAndType/ContextQualifier/theories/Qualifier.v`；第 374 行起；`qual_top_on` 给出该 top qualifier 的实现。

[6] Rocq 对应定义：`/home/sdywolf/Project/UnderLogicAndType/ContextTyping/examples/HypraSecureTyping.v` 第 19 行的 `ni_function_type` 与第 21 行的 `low_function_type` 是这两种类型的结果 capability 契约；它们尚不是原生 `context_ty` 上的函数类型声明。

这两个公开输入的整体集合不满足其第二个参数的 Low 前提。对本文的全定义确定性函数，这两种约束都以式 (1) 为目标。对应结果像契约的等价性及其与完整程序项类型解释的桥接条件在第 5.2 节给出。两个类型均采用普通箭头，输入间的相关性由允许的 capability 表示，不额外要求独立输入。

3 语义域与 Context Logic

capability r 是非空环境集合，其中环境的变量域相同。投影、相容乘积与 Kripke 精化采用论文记号：

$$r\restriction_X = \{\sigma\restriction_X \mid \sigma \in r\}, \qquad r_1 \sqsubseteq r_2 \iff r_1 = r_2\restriction_{\mathrm{dom}(r_1)}.$$

$r_1 \uparrow r_2$ 要求任意跨集合的环境相容，允许一致的重叠绑定；相容时 $r_1 \times r_2 = \{\sigma_1 \cup \sigma_2 \mid \sigma_i \in r_i\}$。

本文用到的逻辑解释如下；公式支持须在 capability 中已绑定：

$$\begin{aligned}
r \models \mathrm{Atom}(\phi) &\iff r\restriction_{\mathrm{FV}(\phi)} = \{\sigma \mid \mathrm{dom}(\sigma) = \mathrm{FV}(\phi),\ \sigma \models \phi\},\\
r \models \boxed{\mathrm{o}}\,P &\iff \exists r'.\ r \subseteq r' \wedge r' \models P,\\
r \models \boxed{\mathrm{u}}\,P &\iff \exists r'.\ r' \subseteq r \wedge r' \models P,\\
r \models \Box P &\iff \exists \sigma.\ \mathrm{dom}(\sigma) = \mathrm{FV}(P) \wedge r\restriction_{\mathrm{FV}(P)} = \{\sigma\} \wedge \{\sigma\} \models P.
\end{aligned}$$

最后一条在 Rocq 中已有等价引理。[7]

对 $X \subseteq \mathrm{dom}(r)$ 和 $\sigma_X \in r\restriction_X$，采用集合 fiber 定义

$$\mathrm{Fiber}(r, X, \sigma_X) = \{\sigma \in r \mid \sigma\restriction_X = \sigma_X\},$$

以及

$$r \models X \triangleright P \iff \forall \sigma_X \in r\restriction_X.\ \mathrm{Fiber}(r, X, \sigma_X) \models \sigma_X(P). \tag{8}$$

$\sigma_X(P)$ 是对整个公式的绑定代入；只检查实际出现的 fiber。右向 fiber 组合定义为 $P \blacktriangleright Q := P \wedge \mathrm{FV}(P) \triangleright Q$。

4 Type Denotational Semantics：从程序项到逻辑公式

4.1 解释接口、guard 与求值结果

沿用论文 Section 4.4 的接口：

$$\boxed{[\![\tau]\!]_\Sigma : \mathrm{Term} \to \mathrm{Formula}, \qquad [\![\Gamma]\!]_\Sigma : \mathrm{Formula}.}$$

$r \models [\![\tau]\!]_\Sigma e$ 表示程序项 e 在 capability r 下满足类型 τ。类型解释同时依赖基本类型上下文 Σ 与程序项 e。

每个类型解释都隐含论文的 guard：

$$P_{\mathrm{guard}} := \mathrm{dom}(\Sigma) \triangleright \mathrm{Atom}\big(\mathrm{Total}(e) \wedge \Sigma \vdash_s e : \lfloor\tau\rfloor \wedge \Sigma \vdash_{\mathrm{WF}} \tau\big).$$

[7] Rocq 声明及证明：`/home/sdywolf/Project/UnderLogicAndType/ContextLogic/theories/FormulaConnectives.v`；第 852 行起；`res_models_persist_iff`。

以下各条右侧均省略与 P_{guard} 的合取。新鲜结果槽由求值图解释：

$$[\![e \hookrightarrow^* \nu]\!]_\Sigma := \mathrm{dom}(\Sigma) \triangleright \mathrm{Atom}(e \hookrightarrow^* \nu).$$

$\forall\nu$ 是论文 context logic 中的新变量 capability 扩展，不是只挑选一次求值结果。基础类型与结果原子的解释在相应基本类型载体中进行；artifact 还显式加入结果基本类型公式。

4.2 基础类型与组合类型的递归解释

采用论文的求值结果优先形式。为方便阅读，$\Sigma_\nu := \Sigma, \nu : \lfloor\tau\rfloor$；新鲜变量均避开输入上下文和待解释类型的自由变量。

$$[\![\{\nu : b \mid \phi\}]\!]_\Sigma e := \forall\nu.\ [\![e \hookrightarrow^* \nu]\!]_\Sigma \Rightarrow (\mathrm{FV}(\phi) \setminus \{\nu\}) \triangleright \boxed{\mathrm{o}}\,\mathrm{Atom}(\phi), \tag{9}$$

$$[\![[\nu : b \mid \phi]]\!]_\Sigma e := \forall\nu.\ [\![e \hookrightarrow^* \nu]\!]_\Sigma \Rightarrow (\mathrm{FV}(\phi) \setminus \{\nu\}) \triangleright \boxed{\mathrm{u}}\,\mathrm{Atom}(\phi), \tag{10}$$

$$[\![\tau_1 \sqcap \tau_2]\!]_\Sigma e := [\![\tau_1]\!]_\Sigma e \wedge [\![\tau_2]\!]_\Sigma e, \tag{11}$$

$$[\![\tau_1 \oplus \tau_2]\!]_\Sigma e := \forall\nu.\ [\![e \hookrightarrow^* \nu]\!]_\Sigma \Rightarrow \big([\![\tau_1]\!]_{\Sigma_\nu}\nu \oplus [\![\tau_2]\!]_{\Sigma_\nu}\nu\big), \tag{12}$$

$$[\![\Box\tau]\!]_\Sigma e := \forall\nu.\ [\![e \hookrightarrow^* \nu]\!]_\Sigma \Rightarrow \Box\big([\![\tau]\!]_{\Sigma_\nu}\nu\big). \tag{13}$$

在 sum 条目中 τ_1, τ_2 的 erasure 须一致，Σ_ν 使用这个共同 erasure。基础 refinement 中，原有绑定的 ν 在解释时统一重命名为新鲜结果槽。上述基础类型对应论文 Definition 4.7 的结果槽形式；sum 与 persistence 使用 Section 5 的结果优先扩展。

4.3 函数类型的解释

函数箭头保持论文 Section 5.2 的结构，先把 e 的函数值结果记为新鲜变量 ν_f。记 $\Sigma_f := \Sigma, \nu_f : \lfloor x : \tau_x \to \tau\rfloor$，$\Sigma_{f,x} := \Sigma_f, x : \lfloor\tau_x\rfloor$，则

$$\begin{aligned}[\![x : \tau_x \to \tau]\!]_\Sigma e := \forall\nu_f.\ &[\![e \hookrightarrow^* \nu_f]\!]_\Sigma \Rightarrow \\ &\forall x.\ [\![\tau_x]\!]_{\Sigma_{f,x}} x \Rightarrow [\![\tau]\!]_{\Sigma_{f,x}}(\nu_f\, x),\end{aligned} \tag{14}$$

$$\begin{aligned}[\![x : \tau_x -\!\!* \tau]\!]_\Sigma e := \forall\nu_f.\ &[\![e \hookrightarrow^* \nu_f]\!]_\Sigma \Rightarrow \\ &\big([\![\tau_x]\!]_{\Sigma_{f,x}} x -\!\!* [\![\tau]\!]_{\Sigma_{f,x}}(\nu_f\, x)\big).\end{aligned} \tag{15}$$

wand 条目中的 Σ_f 使用 separating-arrow 的 erasure（它与普通箭头的基本函数 erasure 相同）。x 是新鲜参数名；wand 的参数绑定按照论文的 binder-aware 解释处理。普通箭头允许参数能力与闭包能力纠缠；separating arrow 通过乘法蕴涵约束其组合。

4.4 整个类型的 fiber 包装

定义 4.1 (观察类型的解释)**.**

$$\boxed{[\![\mathsf{Observe}(X, \tau)]\!]_\Sigma e := X \triangleright \big([\![\tau]\!]_\Sigma e\big).} \tag{16}$$

[8] 其中 $X \subseteq \mathrm{dom}(\Sigma)$，且新鲜结果槽 $\nu \notin X$。代入作用于内部完整公式，包含程序项、类型和 *guard* 所产生的观察。

[8] Rocq 对应定义：`/home/sdywolf/Project/UnderLogicAndType/ContextTyping/theories/FiberResult.v`；第 10 行起；`RTObserve` 及第 75 行的 `fiber_result_denote` 实现结果类型原型的 fiber 包装；这里的程序项级完整类型 denotation 尚未加入原生解释器。

这里必须先按 X 划分输入 fiber，再检查该 fiber 中全部求值结果的类型。结果一致性约束作用于每个输入 fiber 的完整求值结果 capability。以 Low 类型为例，式 (16) 展开为

$$[\![\mathsf{NI}_b(l)]\!]_\Sigma e = \{l\} \triangleright \big(\forall \nu.\ [\![e \hookrightarrow^* \nu]\!]_\Sigma \Rightarrow \Box\big([\![\{\nu : b \mid \top\}]\!]_{\Sigma,\nu:b}\nu\big)\big)\,. \tag{17}$$

式 (17) 同时保留了公开输入的观察域和程序项的完整求值结果。qualifier-dot 作用于 qualifier 的支持域；Observe 将显式观察纪律提升到整个类型。只使用 $\{\nu : b \mid \{l\}.\{\top\}\}$ 并不限制同一 low-fiber 内返回值的数目，因而不能替代该定义。

4.5 Type context 的解释

保持论文 Definition 4.8：

$$\begin{aligned}
[\![\varnothing]\!]_\Sigma &:= \top, & [\![x : \tau]\!]_\Sigma &:= [\![\tau]\!]_\Sigma x,\\
[\![\Gamma_1, \Gamma_2]\!]_\Sigma &:= [\![\Gamma_1]\!]_\Sigma \wedge [\![\Gamma_2]\!]_{\Sigma,\lfloor\Gamma_1\rfloor},\\
[\![\Gamma_1 * \Gamma_2]\!]_\Sigma &:= [\![\Gamma_1]\!]_\Sigma * [\![\Gamma_2]\!]_\Sigma,\\
[\![\Gamma_1 \oplus \Gamma_2]\!]_\Sigma &:= [\![\Gamma_1]\!]_\Sigma \oplus [\![\Gamma_2]\!]_\Sigma.
\end{aligned}$$

语义 typing 的目标形式为 $[\![\Gamma]\!]_\Sigma \models [\![\tau]\!]_{\Sigma,\lfloor\Gamma\rfloor} e$，其中 $\models$ 是论文的公式 entailment。原生声明式判断仍写为 $\Gamma \vdash e : \tau$，语义 entailment 与声明式 typing derivation 是两个不同的判断。

5 类型解释与结果 capability 性质

5.1 取值一致性与 noninterference

以下是逻辑公式而非类型构造：

$$B_\nu := \boxed{\mathrm{o}}\,\mathrm{Atom}(\{\nu\}.\{\top_\varnothing\}), \qquad \mathsf{Low}(\nu) := B_\nu * B_\nu.$$

[9] 对整数值域，$r \models B_\nu$ 等价于 $\nu \in \mathrm{dom}(r)$。[10]

命题 5.1 (结果 capability 上的 Low，已证明)**.**

$$r \models \mathsf{Low}(\nu) \iff \nu \in \mathrm{dom}(r) \wedge \forall \sigma, \sigma' \in r.\ \sigma(\nu) = \sigma'(\nu). \tag{18}$$

该等价在已有逻辑层证明。[11]

非空性保证这也对应一个 singleton 结果投影。[12]

[9] Rocq 对应定义：`/home/sdywolf/Project/UnderLogicAndType/ContextLogic/theories/FormulaNoninterference.v`；第 19 行起；`FBound` 与第 21 行的 `FLow` 是这两个公式的 Rocq 定义。

[10] Rocq 声明及证明：`/home/sdywolf/Project/UnderLogicAndType/ContextLogic/theories/FormulaNoninterference.v`；第 41 行起；`FBound_spec`。

[11] Rocq 声明及证明：`/home/sdywolf/Project/UnderLogicAndType/ContextLogic/theories/FormulaNoninterference.v`；第 79 行起；`FLow_spec`。

[12] Rocq 声明及证明：`/home/sdywolf/Project/UnderLogicAndType/ContextTyping/theories/FiberResult.v`；第 25 行起；`low_result_singleton`。

命题 5.2 (公式层 persistence 对应，已证明)**.** $r \models \mathsf{Low}(\nu) \iff r \models \Box B_\nu$。[13]

原生类型 Low_b 的引入方向也有桥接：基本结果类型的 denotation 加上 $\mathsf{Low}(\nu)$ 能推出 $[\![\mathsf{Low}_b]\!]_\Sigma \nu$。[14]

命题 5.3 (结果 capability 上的 NI，已证明)**.** 对于 $l \neq \nu$，

$$r \models l \triangleright \mathsf{Low}(\nu) \iff \{l, \nu\} \subseteq \mathrm{dom}(r) \wedge \\ \forall \sigma, \sigma' \in r.\ \sigma(l) = \sigma'(l) \Longrightarrow \sigma(\nu) = \sigma'(\nu).$$

该等价刻画固定公开输入后的输出一致性。[15]

公式 $l \triangleright \mathsf{Low}(\nu)$ 对应的逻辑定义为 *FNoninterference*。[16]

式 (17) 通过程序求值图建立输入 fiber 与结果 capability 的联系。从该类型解释到上述结果性质的桥接，需要求值图扩展、类型 guard 与观察操作的相容性。

5.2 结果像契约

对全定义确定性整数函数，定义

$$\mathrm{Img}_f(r) := \{\sigma[\nu \mapsto f(\sigma(h), \sigma(l))] \mid \sigma \in r\}.$$

[17] 记 $P_{\mathrm{all}} := B_h \wedge B_l$，$P_{\mathrm{low}} := B_h \wedge \mathsf{Low}(l)$，以及

$$C_{\mathsf{NI}}(f) := \forall r.\ r \models P_{\mathrm{all}} \Longrightarrow \mathrm{Img}_f(r) \models l \triangleright \mathsf{Low}(\nu), \\ C_{\mathrm{pres}}(f) := \forall r.\ r \models P_{\mathrm{low}} \Longrightarrow \mathrm{Img}_f(r) \models \mathsf{Low}(\nu).$$

[18] C_{NI}、C_{pres} 分别刻画任意输入 capability 的 low-fiber 一致性与公开输入固定时的一致性保持。

命题 5.4 (辅助契约的准确性，已证明)**.** 对于本文函数片段，

$$C_{\mathsf{NI}}(f) \iff C_{\mathrm{pres}}(f) \iff \forall h_1, h_2, l.\ f(h_1, l) = f(h_2, l).$$

两项分别由已有的契约等价定理支持。[19]

[13] Rocq 声明及证明：`/home/sdywolf/Project/UnderLogicAndType/ContextTyping/theories/FiberResult.v`；第 48 行起；`low_result_persistence`。

[14] Rocq 声明及证明：`/home/sdywolf/Project/UnderLogicAndType/ContextTyping/theories/LowContextType.v`；第 16 行起；`low_context_ty_intro`。这里只证明引入方向，不是完整的双向类型编码。

[15] Rocq 声明及证明：`/home/sdywolf/Project/UnderLogicAndType/ContextTyping/theories/FiberResult.v`；第 98 行起；`ni_result_denote`；第 108 行的 `ni_result_spec`；第 90 行的 `padded_low_result_eq`。

[16] Rocq 对应定义：`/home/sdywolf/Project/UnderLogicAndType/ContextLogic/theories/FormulaNoninterference.v`；第 22 行起；`FNoninterference` 定义 low-fiber 输出一致性公式。

[17] Rocq 对应定义：`/home/sdywolf/Project/UnderLogicAndType/ContextLogic/examples/HypraSecure.v`；第 55 行起；`outputs` 构造输入 capability 的完整输出像。

[18] Rocq 对应定义：`/home/sdywolf/Project/UnderLogicAndType/ContextTyping/theories/FiberResult.v` 第 118 行的 `has_fiber_function_type`，以及 `/home/sdywolf/Project/UnderLogicAndType/ContextTyping/examples/HypraSecureTyping.v` 第 19、21 行的 `ni_function_type`、`low_function_type`；后两者使用该通用结果 capability 契约实例化本文的两个函数规格。

[19] Rocq 声明及证明：`/home/sdywolf/Project/UnderLogicAndType/ContextTyping/examples/HypraSecureTyping.v`；第 53 行起；`ni_function_type_iff`；第 80 行的 `low_function_type_iff`，二者均是结果像接口的定理。

另有 $C_{\mathsf{NI}}(f) \Longleftrightarrow \exists g : \mathbb{Z} \to \mathbb{Z}.\ \forall h, l.\ f(h,l) = g(l)$。[20]

程序项解释与结果像契约的对应。 令论文的单位 capability $1 := \{\varnothing\}$。在闭合全定义确定性函数片段中，令 e_f 为代表整数函数 f 的闭合函数程序项。两层语义的桥接需要建立以下等价关系：

$$1 \models [\![T_{\mathsf{NI}}]\!]_{\varnothing} e_f \Longleftrightarrow C_{\mathsf{NI}}(f), \qquad 1 \models [\![T_{\mathrm{pres}}]\!]_{\varnothing} e_f \Longleftrightarrow C_{\mathrm{pres}}(f).$$

左侧解释的是函数程序项，右侧解释的是其整数函数行为；1 不是空环境集合。现有 Rocq 证明覆盖右侧结果像契约；上述程序项解释的双向桥接尚未纳入验证范围。

6 整数程序实例

Hypra Figure 2 的 `secure`[3] 及其泄漏变体定义如下，Rocq 对应程序位于 `ContextLogic/examples/HypraSecure.v`：

```
first_if(h,l) := if h > 0 then 2*l else l;
secure(h,l) := let o = first_if(h,l) in
               if h <= 0 then o+l else o.
```

两个条件依次执行，秘密输入 h 可取任意整数。当 $h > 0$，`secure` 返回 $2l$；当 $h \le 0$，它返回 $l + l = 2l$。[21] 故 `secure` 满足前节定义的结果像契约 C_{NI} 和 C_{pres}；这为所提出的 T_{NI} 与 T_{pres} 提供了语义实例。[22]

`first_if` 不满足 C_{NI}：[23]

$$\texttt{first_if}(1,3) = 6 \neq 3 = \texttt{first_if}(-1,3).$$

该反例由公开输入相同、秘密分支不同的两次执行构成。

以下三项用于检查规格的强弱。

1. 若结果要求全局 $\mathsf{Low}(\nu)$，`secure` 会被错误拒绝：$l = 0$ 与 $l = 1$ 分别产生 0 与 2。NI 只要求同低输入的一致性。
2. 若改为 $\mathsf{Observe}(\{h,l\}, \square\{\nu : \mathsf{int} \mid \top\})$，确定性泄漏函数 $f(h,l) = h$ 也会通过：固定全部输入的 fiber 内输出当然一致。因此观察域是安全策略的一部分。
3. 常数函数满足 NI。该类型没有输出 coverage 的要求，不应强制它覆盖所有整数。

其中 `secure` 的接受、`first_if` 的拒绝、全局取值一致性要求的错误拒绝与常数函数的接受，均在结果像契约层有 Rocq 证明。[24] 第二项是观察域选择的语义反例，其成立可由式 (16) 展开检

[20] Rocq 声明及证明：`/home/sdywolf/Project/UnderLogicAndType/ContextTyping/examples/HypraSecureTyping.v`；第 102 行起；`ni_iff_low_summary`。

[21] Rocq 声明及证明：`/home/sdywolf/Project/UnderLogicAndType/ContextLogic/examples/HypraSecure.v`；第 33 行起；`secure_result`，即所有整数输入下返回 $2l$ 的计算等式。

[22] Rocq 声明及证明：`/home/sdywolf/Project/UnderLogicAndType/ContextTyping/examples/HypraSecureTyping.v`；第 110 行起；`secure_has_ni_type`；第 113 行的 `secure_has_low_type`。

[23] Rocq 声明及证明：`/home/sdywolf/Project/UnderLogicAndType/ContextTyping/examples/HypraSecureTyping.v`；第 116 行起；`first_if_rejected`。

[24] Rocq 声明及证明：`/home/sdywolf/Project/UnderLogicAndType/ContextTyping/examples/HypraSecureTyping.v`；第 122 行起；`constant_has_ni_type`；第 126 行的 `secure_not_globally_low`。观察域扩大为 $\{h,l\}$ 的反例尚无独立的 Rocq 定理。

查。

7 由类型性质导出的 VC

对有限精确路径摘要 $\{(G_i, E_i)\}_i$，每条路径给出输入守卫和返回表达式。在固定低输入的 fiber 中，两次执行共享 l，秘密输入与所选路径均可不同。因此路径对的 VC 是

$$\forall h_1, h_2, l. \quad G_i(h_1, l) \wedge G_j(h_2, l) \Longrightarrow E_i(h_1, l) = E_j(h_2, l). \tag{19}$$

不增加 $h_1 = h_2$ 或 $i = j$ 的约束。[25]

原两个条件产生四条符号路径：

第一条件	第二条件	守卫	结果
true	true	$h > 0 \wedge h \leq 0$	$3l$（不可达）
true	false	$h > 0$	$2l$
false	true	$h \leq 0$	$l + l$
false	false	$h \leq 0 \wedge h > 0$	l（不可达）

两两配对生成 16 个 VC；其中 12 个由不可达路径的矛盾守卫成立。[26]

不同可达分支之间的关键 VC 为

$$h_1 > 0 \wedge h_2 \leq 0 \Longrightarrow 2l = l + l.$$

`first_if` 的相应 VC 则是 $2l = l$，由 $(1, -1, 3)$ 反驳。[27] 两个分支内部的表达式都不提秘密，也不能省略这个跨分支检查。

命题 7.1 (路径 VC 与结果像契约，已实现)**.** *假设路径摘要对原函数 sound，且每个输入至少满足一个路径守卫。则式 (19) 对所有路径对成立，当且仅当 $C_{\mathsf{NI}}(f)$。*[28]

`vcgen_correct` 实现此双向结论；两个例子均提供路径正确性和输入覆盖证明。[29]

`vcgen` 的输入是具备 soundness 与 coverage 证明的有限路径摘要，输出为全部路径配对义务。两个实例中的线性整数算术 VC 由 `lia` 验证。

[25] Rocq 对应定义：`/home/sdywolf/Project/UnderLogicAndType/ContextTyping/examples/HypraSecureVC.v`；第 12 行起；`path`、第 25 行的 `path_pair_vc` 与第 30–32 行的 `vcgen/vcs_valid` 实现有限路径摘要及其成对 VC。

[26] Rocq 声明及证明：`/home/sdywolf/Project/UnderLogicAndType/ContextTyping/examples/HypraSecureVC.v`；第 96 行起；`secure_generates_sixteen_vcs`；第 99 行的 `secure_vcs_valid`。12 个 vacuous VC 在后者的分支配对证明中由算术矛盾处理，没有另行命名的计数定理。

[27] Rocq 声明及证明：`/home/sdywolf/Project/UnderLogicAndType/ContextTyping/examples/HypraSecureVC.v`；第 116 行起；`secure_opposite_branches_vc`；第 141 行的 `first_if_vcs_rejected` 和第 148 行的 `first_if_mixed_vc_counterexample`。

[28] Rocq 声明及证明：`/home/sdywolf/Project/UnderLogicAndType/ContextTyping/examples/HypraSecureVC.v`；第 45 行起；`vcgen_correct`。

[29] Rocq 声明及证明：`/home/sdywolf/Project/UnderLogicAndType/ContextTyping/examples/HypraSecureVC.v`；第 73 行起；`secure_paths_sound`。第 85 行：`secure_paths_cover`。第 108 行：`secure_typed_via_vcs`。`first_if` 的路径 soundness/coverage 分别在第 125、134 行。

8 形式化结果与实现范围

Rocq 开发验证了结果 capability 上的取值一致性、noninterference 刻画、结果像契约的准确性，以及精确有限路径摘要下的 VC 正确性。类型语法与程序项解释提供了这些结果的类型层组织方式。表中列出各组成部分的实现状态。

组成部分	实现状态
Context Types 基础语法与结果优先解释	原 artifact 已实现；CoreLang 的基本类型包括 unit、bool、nat。
qualifier-dot	`SupportPadding.v` 实现支持域扩充。
整个类型的 $\mathsf{Observe}(X, \tau)$	本文定义其语法、erasure、良构条件与程序项 denotation；原 `context_ty` 尚未包含该构造。
Low 与 NI 的 capability 刻画	`FiberResult.v` 给出相应证明；`LowContextType.v` 给出 persistent 类型的引入桥接。
整数程序与路径 VC	`HypraSecureTyping.v` 和 `HypraSecureVC.v` 在结果像契约层完成验证。
完整程序项类型验证	程序项 denotation 与结果像契约的双向桥接、整数 CoreLang 编码和相应 `has_context_type` 推导属于进一步的形式化工作。

参考文献

[1] *Context Types: Bunched Modalities for Unifying Safety and Reachability.* 类型语法：Figure 2；denotation：Section 4.4；persistence 与结果优先扩展：Section 5。

[2] *Fiber-Guided Support Refinement and Type Negation.* 支持域扩充与右向 fiber 操作，第 1–2 节。

[3] Thibault Dardinier, Ruyi Li, Peter Müller. *Hypra: A Deductive Program Verifier for Hyper Hoare Logic.* OOPSLA 2024，Figure 2 的 `secure`；本文使用对应的整数程序实例。